Homework 2

Making Your Model Talk

Hands-on AI · SEMFE, NTUA

Deadline: 2 weeks from assignment date

Overview

In this assignment you will take the best-performing model you produced in Homework 1 and give it a voice. You will build a domain-aware conversational AI agent that can both answer natural language questions about your domain and make predictions using your trained model – all within a single unified system.

> **Important:** This assignment builds directly on Homework 1. You will need your saved `best_model.pkl` and `scaler.pkl` from HW1. If these files are missing or incompatible, you will need to retrain your pipeline before proceeding. Do not start this assignment without verifying that your HW1 artifacts load correctly.

The mandatory tasks are **Tasks 1 through 4**. Tasks 5 and 6 are optional and yield **bonus marks**.

> **LLM API Requirement:** This assignment requires calls to a Large Language Model. Use a cloud LLM API – OpenAI, Anthropic, or Google Gemini all provide free tiers sufficient for this coursework. **Do not attempt to run a local LLM** (e.g. via Ollama) as inference will be prohibitively slow and unreliable for development and testing.

Required Project Structure

Submit your work as a **GitHub repository URL**. The repository must extend your HW1 repository (or reference it clearly) and follow this structure:

```
hw2/
|-- main.py                    # Entry point: starts the
   FastAPI application
|-- src/
|    |-- rag.py                # RAG system: document loading,
   vector store, retrieval
|    |-- tools.py              # LangGraph tool definitions
|    |-- agent.py              # LangGraph agent definition and
    graph
|    `-- api.py                # FastAPI /chat endpoint
|-- data/
|    |-- documents/            # Domain documents for the
   knowledge base
|    `-- vector_store/         # Persisted vector store (
   ChromaDB or FAISS)
|-- models/
```

```
|   |-- best_model.pkl          # Carried over from HW1
|   `-- scaler.pkl              # Carried over from HW1
|-- requirements.txt
`-- README.md
```

Task 1. Knowledge Base & RAG System

Build a domain-specific knowledge base from documents related to your HW1 domain. This knowledge base will allow your agent to answer factual and conceptual questions about the domain using Retrieval-Augmented Generation (RAG).

1.1. Document Collection

Gather at least **5 documents** relevant to your domain. These can be:

- Wikipedia articles or summaries
- Research paper abstracts or introductions
- Domain-specific reports, blog posts, or documentation
- Any publicly available text that covers concepts in your domain

Store the raw documents as `.txt` or `.pdf` files inside `data/documents/`.

> **Example:**
> *Domain* – E-commerce (from HW1).
> *Documents collected* – (1) Wikipedia article on "Online shopping", (2) Abstract of a paper on customer churn prediction, (3) A Kaggle blog post on purchase intent modelling, (4) A summary of e-commerce return rate statistics, (5) An article on product recommendation systems.
> These 5 documents give the agent background knowledge to answer questions like "What factors typically influence whether a customer returns a product?"

1.2. Vector Store

Chunk the documents, embed them using an embedding model, and store the resulting vectors in a persistent vector store. The vector store must be saved to disk so it does not need to be rebuilt on every startup.

- Use **ChromaDB** or **FAISS** as the vector store
- Use any embedding model available via LangChain (e.g. `OpenAIEmbeddings`, `HuggingFaceEmbedding`
- Use a chunk size and overlap that is appropriate for your document lengths

1.3. Retrieval

Implement a retrieval function that, given a user query, returns the most relevant document chunks. This function will be used as a tool by the agent in Task 3.

Example:
User query – "What is the average return rate in e-commerce?"
Retrieved chunks – Two passages from the return rate statistics document and one from the Wikipedia article, ranked by semantic similarity to the query.
Retrieval function output – A single string concatenating the top-3 retrieved passages, to be passed to the LLM as context.

Task 2. Model as a Tool

Wrap your HW1 best model as a callable tool that the LangGraph agent can invoke to make predictions. The tool must load the saved `best_model.pkl` and `scaler.pkl` from HW1, apply the same preprocessing steps used during training, and return a human-readable prediction.

Preprocessing consistency: The tool must apply **exactly the same** transformations as your HW1 pipeline – scaling, encoding, and any feature engineering. If these transformations are not applied before calling the model, predictions will be incorrect. Load the saved `scaler.pkl` rather than re-fitting a new one.

The tool must:

- Accept input features as a dictionary or JSON string from the agent
- Apply preprocessing (scaling, encoding, feature engineering) using the saved artifacts from HW1
- Return the model's prediction and probability as a human-readable string

Example:
Tool name – `predict_return`
Tool description – "Predicts whether an e-commerce transaction will result in a product return. Input must be a JSON string with fields: customer_age, price, quantity, discount_pct, region, product_category, is_member, days_since_last_purchase."
Tool input (from agent) – `'{"customer_age": 34, "price": 89.99, "quantity": 2, "discount_pct": 10, "region": "North", "product_category": "Electronics", "is_member": 1, "days_since_last_purchase": 15}'`
Tool output (to agent) – `"Prediction: Not Returned (probability of return: 12.3%)"`

Task 3. Conversational AI Agent

Build a **LangGraph agent** that integrates the RAG system and the prediction tool into a single conversational interface. The agent must be capable of handling both types of requests – domain knowledge questions and prediction requests – within the same conversation.

3.1. Agent Architecture

Your agent must have access to exactly **two tools**:

Tool	Description
Retrieval Tool	Calls the RAG retrieval function from Task 1. Used when the user asks a factual or conceptual question about the domain.
Prediction Tool	Calls the HW1 model from Task 2. Used when the user wants a prediction given specific input values.

The agent must decide autonomously which tool to call (or whether to call any tool) based on the user's message. It must **not** require the user to explicitly say "use the RAG tool" or "make a prediction" – the agent should infer the appropriate action from the natural language input.

3.2. Conversation Memory

The agent must maintain **conversation memory** across turns within a session. A user should be able to ask a follow-up question that references a previous message, and the agent should respond correctly.

Example conversation:
User: "What factors typically cause customers to return products?"
Agent: [calls Retrieval Tool] "Based on the available literature, the main factors are high discount rates, product-quality mismatches, and impulse purchases. Customers who receive large discounts tend to have higher return rates. . . "

User: "Interesting. Would a customer aged 34 who bought Electronics for \$90 with a 10% discount likely return it?"
Agent: [calls Prediction Tool] "Based on your HW1 model, this transaction has a 12.3% probability of being returned – so likely not returned. This is consistent with what the research suggests about moderate discounts."

User: "What if the discount was 30% instead?"
Agent: [calls Prediction Tool with updated discount] "With a 30% discount, the return probability rises to 41.2% – the model now considers a return more plausible, which aligns with the literature on high-discount return rates we discussed earlier."

Note on memory: Conversation memory only needs to persist **within a single session**. You do not need to implement persistent memory across separate API calls or application restarts.

Task 4. FastAPI Endpoint

Expose your agent via a REST API using **FastAPI**. Create a `/chat` POST endpoint that accepts a user message and returns the agent's response.

Use `Pydantic` to define the input and output schemas. The endpoint must:

- Accept a `message` (string) and a `session_id` (string) in the request body
- Route the message to the LangGraph agent

- Return the agent's response as a string
- Use the `session_id` to maintain separate conversation histories for different users or sessions

Run the application with `uvicorn` and verify it through the auto-generated Swagger UI at `http://127.0.0.1:8000/docs`.

> **Example:**
> *Request* – `POST /chat` with body `{"message": "Will a 45-year-old customer buying Books for $25 with no discount return the item?", "session_id": "user_001"}`
> *Response* – `{"response": "Based on your model, this transaction has a 6.1% probability of being returned - very unlikely. Books purchased at full price by older customers show historically low return rates."}`

Task 5. Additional Tool (Bonus)

Extend your agent with a **third tool** of your choice. The tool must add genuine value to the agent's capabilities and be clearly documented in the README. Some ideas:

- **Web search** – allows the agent to retrieve up-to-date information beyond what is in the knowledge base
- **Dataset statistics tool** – queries the HW1 dataset to return summary statistics on demand (e.g. average price, class distribution)
- **Calculator / unit converter** – useful for domains involving numerical reasoning
- **CSV lookup tool** – looks up a specific row or subset of rows from the HW1 dataset given filter criteria

> **Example:**
> *Tool name* – `dataset_stats`
> *Tool description* – "Returns summary statistics for any numerical column in the e-commerce dataset. Input: column name as a string."
> *User:* "What is the average price in the dataset?"
> *Agent:* [calls dataset_stats with input "price"] "The average transaction price in the dataset is $94.3, with a median of $72.1 and a standard deviation of $61.8."

Task 6. Streaming Output (Bonus)

Add **streaming support** to the `/chat` endpoint so that the agent's response is streamed token-by-token to the client rather than returned all at once. Use FastAPI's `StreamingResponse` with Server-Sent Events (SSE).

> **Example:**
> The client sends a POST request to `/chat/stream`. Instead of waiting for the full response, the client receives tokens progressively as the LLM generates them – similar to how ChatGPT displays responses. The streaming endpoint should be separate from the standard `/chat` endpoint so both remain available.

Deliverables Checklist

File	Description	Required
`main.py`	Starts the FastAPI application	✓
`src/rag.py`	RAG system implementation	✓
`src/tools.py`	Tool definitions	✓
`src/agent.py`	LangGraph agent	✓
`src/api.py`	FastAPI `/chat` endpoint	✓
`data/documents/`	At least 5 domain documents	✓
`data/vector_store/`	Persisted vector store	✓
`models/best_model.pkl`	Best model from HW1	✓
`models/scaler.pkl`	Scaler from HW1	✓
`requirements.txt`	All Python dependencies	✓
`README.md`	Documentation (see below)	✓
Third tool in `src/tools.py`	Additional agent tool	⋆ Bonus
`/chat/stream` endpoint	Streaming SSE response	⋆ Bonus

README Requirements

Your `README.md` must contain the following sections:

1. **System Overview** – A short description of the agent, the domain, and what it can do.
2. **Architecture** – A description of the two (or three) tools, the LangGraph graph structure, and how the agent decides which tool to call.
3. **Knowledge Base** – What documents were collected, why they were chosen, and what kinds of questions the RAG system can answer.
4. **HW1 Model Integration** – Which model was carried forward from HW1, what the prediction tool does, and what input it expects.
5. **Example Conversations** – At least 2 example conversation turns demonstrating (a) a RAG retrieval response and (b) a prediction response.
6. **Installation & Execution** – Step-by-step commands to clone the repo, install dependencies, and start the FastAPI server.
7. **Example API call** – A `curl` or Python `requests` example showing how to call the `/chat` endpoint.

Key Reminders

Use a cloud LLM API: Do not run a local LLM. Use OpenAI, Anthropic, or Gemini with a free-tier API key. Store your API key as an environment variable – never hardcode it in your source code or commit it to GitHub.

HW1 artifacts must match: The `scaler.pkl` and `best_model.pkl` in your HW2 repository must be identical to those produced by your HW1 pipeline. Do not refit the scaler or retrain the model. The prediction tool in this assignment must produce the same results as the `/predict` endpoint in HW1.

Save the vector store: Build the vector store once and persist it to `data/vector_store/`. The application must load the existing store on startup rather than rebuilding it every time. Rebuilding on every startup is slow and consumes API credits unnecessarily.

Good luck! Your agent is the bridge between your HW1 model and the real world.